

MellonBank – Project

Βελεγράκης Κωνσταντίνος – belekostac@gmail.com



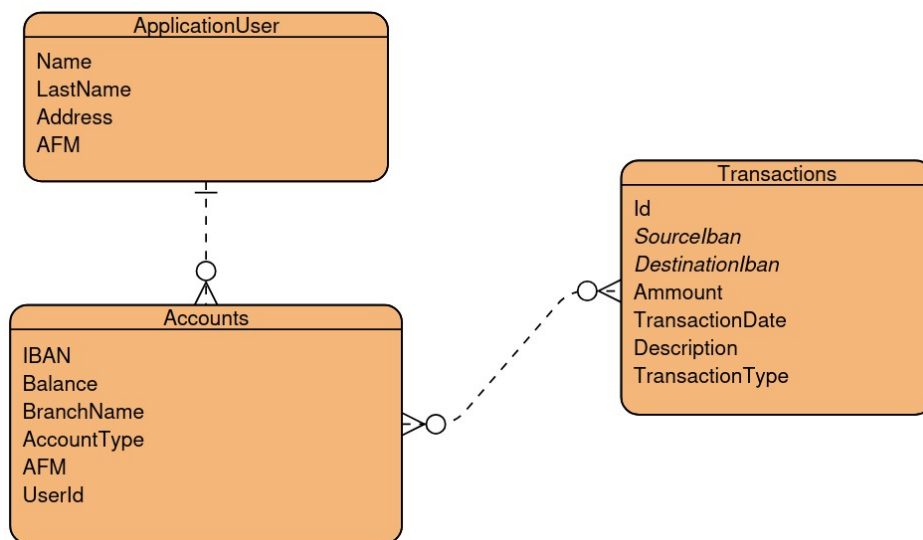
Table of Contents

Τεχνολογικό Stack.....	2
Διάγραμμα ER Βάσης Δεδομένων.....	2
Models.....	3
Controllers.....	5
StaffController.....	5
CustomerController.....	5
HomeController.....	6
AccountController.....	6
Currency Service.....	6
ViewModels.....	7
Views.....	7
Οδηγίες Εγκατάστασης.....	8

Τεχνολογικό Stack

Framework	ASP.NET Core MVC
Γλώσσα	C#
Βάση Δεδομένων	MS SQL Server
Frontend	Razor Views, Javascript
Ασφάλεια	ASP.NET Core Identity

Διάγραμμα ER Βάσης Δεδομένων



Σημειώνουμε πως στο παραπάνω διάγραμμα, δεν καταγράφονται τα πεδία του πίνακα ApplicationUser που κληρονομούνται από τον *IdentityUser* του ASP.NET Core Identity Framework. Κάποιες από αυτές είναι:

- UserId
- UserName
- PhoneNumber
- Email
- PasswordHash

Επιπλέον, μέσω του παραπάνω Framework διαχειριζόμαστε και την ανάθεση των **ρόλων** στους χρήστες (Staff/Customer), γιατί και δεν υπάρχει στο διάγραμμα.

Models

Τα τρία μοντέλα/entities της εφαρμογής είναι:

- ◆ ApplicationUser: Όπως είπαμε παραπάνω, είναι η επέκταση του *IdentityUser*. Όλα τα επιπλέον πεδία είναι τύπου *string*. Επιπλέον, για την υλοποίηση της σχέσης One-to-Many, προσθέτουμε και το πεδίο *BankAccounts* τύπου *ICollection<BankAccount>*.

```
23 references
public class ApplicationUser : IdentityUser
{
    // IdentityUser already supports fields:
    // - Id
    // - UserName
    // - PhoneNumber
    // - Email
    // - PasswordHash

    10 references
    public string? Name {get;set;}
    10 references
    public string? LastName {get;set;}
    6 references
    public string? Address {get;set;}
    21 references
    public string? AFM {get;set;}
    0 references
    public ICollection<BankAccount>? BankAccounts {get;set;}
}
```

- ◆ BankAccount: Τα πεδία του μοντέλου για τους τραπεζικούς λογαριασμούς φαίνονται παρακάτω. Χρησιμοποιούμε τον τύπο *decimal* για το τραπεζικό υπόλοιπό του (λόγω της μεγαλύτερης ακρίβειάς του υποδεικνύεται για χρήματα), με ακρίβεια 2 δεκαδικών ψηφίων. Το πεδίο IBAN είναι το Primary Key του πίνακα, ενώ η One-To-Many σχέση που έχει με τους χρήστες, επισφραγίζεται με το τελευταίο πεδίο και το *data annotation*

```

10 references
public class BankAccount {
    [Key]
    10 references
    public required string IBAN {get;set;}

    [Column(TypeName = "decimal(18,2)")] // Setting the
    9 references
    public decimal Balance {get;set;} = 0;
    5 references
    public string? BranchName {get;set;}
    8 references
    public string? AccountType {get;set;}

    7 references
    public required string AFM { get; set; }

    [ForeignKey("UserId")]
    2 references
    public required string UserId {get;set;}

    1 reference
    public virtual ApplicationUser? User { get; set; }
}

```

- ◆ Transaction: Τα παρακάτω πεδία τα χρησιμοποιούμε για την πλήρη καταγραφή και αρχειοθέτηση κάθε χρηματικής κίνησης εντός του συστήματος. Κάθε συναλλαγή ταυτοποιείται μοναδικά από ένα Id και συνδέει έναν λογαριασμό πηγής (SourceIban) με έναν λογαριασμό προορισμού (DestinationIban). Το πεδίο Amount καθορίζει το ύψος της συναλλαγής, ενώ η ημερομηνία και ώρα εκτέλεσης καταγράφονται αυτόματα στο TransactionDate για λόγους ελέγχου και διαφάνειας. Τέλος, το μοντέλο πλαισιώνεται από μια σύντομη περιγραφή (Description) που αιτιολογεί την κίνηση, καθώς και από το TransactionType, το οποίο κατηγοριοποιεί τη συναλλαγή (π.χ. μεταφορά ή κατάθεση), διευκολύνοντας έτσι την οργάνωση του ιστορικού για τον χρήστη.

```

4 references
public class Transaction
{
    [Key]
    0 references
    public int Id { get; set; }
    [Required]
    4 references
    public required string SourceIban { get; set; }
    [Required]
    3 references
    public required string DestinationIban { get; set; }
    [Required]
    2 references
    public decimal Amount { get; set; }
    3 references
    public DateTime TransactionDate { get; set; } = DateTime.Now;
    2 references
    public string? Description { get; set; }
    1 reference
    public string TransactionType { get; set; }
}

```

Controllers

Για την εφαρμογή μας, οργανώσαμε τους Controllers με γνώμονα τον διαχωρισμό των ρόλων και την ευανάγνωστη δομή του κώδικα.

StaffController

Στον υποφάκελο των Controllers, Controllers/Staff, υπάρχει εκεί μία partial κλάση StaffController, “σπασμένη” σε τρία αρχεία.

- └─ Staff
 - └─ AccountMngmnt.cs: Προσθήκη/Επεξεργασία/Διαγραφή λογαριασμών
 - └─ CustomerMngmnt.cs: Προσθήκη/Επεξεργασία/Διαγραφή πελατών
 - └─ StaffController.cs: Λίστα πελατών, Λεπτομέρειες πελάτη και Προσθήκη προσωπικού

CustomerController

Αποτελεί τον πυρήνα της εμπειρίας του πελάτη. Είναι προστατευμένος με το attribute `[Authorize(Roles = "Customer")]`, διασφαλίζοντας ότι μόνο πιστοποιημένοι πελάτες έχουν πρόσβαση. Υλοποιεί τη λογική των τραπεζικών εμβασμάτων χρησιμοποιώντας Database Transactions. Έτσι, διασφαλίζεται

το atomicity της συναλλαγής, ενώ γίνονται αυστηροί έλεγχοι για επαρκές υπόλοιπο και έγκυρα ποσά.

Παρέχει στον χρήστη πλήρη εικόνα των κινήσεών του, φιλτράροντας τις συναλλαγές με βάση το IBAN του.

Ασφάλεια: Περιλαμβάνει λειτουργία αλλαγής κωδικού πρόσβασης μέσω του UserManager.

HomeController

Λειτουργεί ως ο κεντρικός Controller της εφαρμογής. Κατά την είσοδο, ελέγχει αυτόματα την κατάσταση σύνδεσης και τον ρόλο του χρήστη, ανακατευθύνοντάς τον στο αντίστοιχο Dashboard (Staff ή Customer). Διαχειρίζεται την αρχική σελίδα και τη γενική ροή σύνδεσης/αποσύνδεσης.

AccountController

Διαχειρίζεται το Login και το Logout των χρηστών, αξιοποιώντας τον SignInManager για την ασφαλή ταυτοποίηση των στοιχείων στη βάση δεδομένων.

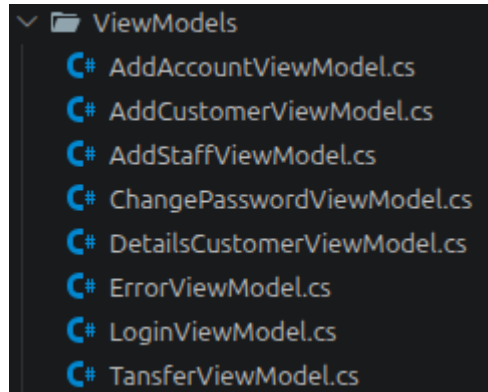
Currency Service

Στον CustomerController, χρησιμοποιούμε το Currency Service για να εμπλουτίσουμε την εμπειρία του χρήστη με δεδομένα σε πραγματικό χρόνο.

- Λειτουργία: Το service υλοποιεί το interface ICurrencyService και είναι υπεύθυνο για την επικοινωνία με το εξωτερικό API [Fixer.io](https://fixer.io/).
- Υλοποίηση: Χρησιμοποιούμε τον IHttpClientFactory για τη δημιουργία ασφαλών HTTP κλήσεων. Το service ανακτά την τρέχουσα ισοτιμία ευρώ-δολαρίου (EUR/USD), επιτρέποντας στον πελάτη να βλέπει την αξία των καταθέσεών του σε διεθνές νόμισμα.
- Error Handling: Έχει προβλεφθεί μηχανισμός try-catch και μια default τιμή (fallback στο 1.1), ώστε αν το εξωτερικό API είναι μη διαθέσιμο, η εφαρμογή να συνεχίσει να λειτουργεί απρόσκοπτα χωρίς να "κρασάρει" το Dashboard του πελάτη.

ViewModels

Πριν προχωρήσουμε στα Views και στο frontend της εφαρμογής, είναι σημαντικό να αναφέρουμε τα ViewModels. Χρησιμοποιήθηκαν για την ασφαλή μεταφορά δεδομένων μεταξύ Views και Controllers, ενσωματώνοντας Data Annotations.



- AddCustomer & AddStaff: Διασφαλίζουν την ορθότητα στοιχείων μέσω Regular Expressions (π.χ. ΑΦΜ 9 ψηφίων, τηλέφωνο 10 ψηφίων).
- TransferViewModel: Διαχειρίζεται τα εμβάσματα, περιλαμβάνοντας λίστα λογαριασμών (MyAccounts) για δυναμική επιλογή από τον χρήστη και πεδία για ποσό και περιγραφή.
- AddAccountViewModel: Επιβάλλει έλεγχο εύρους (Range) στο αρχικό υπόλοιπο, εμποδίζοντας αρνητικές τιμές.
- ChangePasswordViewModel: Χρησιμοποιεί το attribute [Compare] για επιβεβαίωση του νέου κωδικού και επιβάλλει ελάχιστο μήκος χαρακτήρων για ασφάλεια.
- DetailsCustomerViewModel: Συγκεντρώνει το πλήρες προφίλ του πελάτη, τη λίστα των λογαριασμών του και την τρέχουσα ισοτιμία USD σε μία ενιαία προβολή.

Views

- Δομή & Οργάνωση: Ακολουθήθηκε η standard πρακτική του MVC, με τα Views να ομαδοποιούνται σε υποφακέλους βάσει του Controller τους. Οι σελίδες διαχείρισης συγκεντρώθηκαν στο Views/Staff/, ενώ οι τραπεζικές λειτουργίες στο Views/Customer/.
- Layout & Consistency: Το κεντρικό αρχείο _Layout.cshtml διασφαλίζει ενιαία εμφάνιση σε όλο το site. Το navigation menu αλλάζει δυναμικά: Το κουμπί της αποσύνδεσης εμφανίζεται μόνο εφόσον έχει συνδεθεί ο

χρήστης και στο Προσωπικό εμφανίζεται η επιλογή δημιουργίας και άλλων λογαριασμών προσωπικού

- **Φόρμες & Validation:** Χρησιμοποιήθηκαν Tag Helpers (π.χ. asp-for) για τη σύνδεση με τα ViewModels. Η επικύρωση δεδομένων γίνεται άμεσα στο frontend (Client-side validation) μέσω των βιβλιοθηκών jQuery Validation, δίνοντας άμεσα feedback στον χρήστη κατά την προσπάθειας υποβολής της φόρμας.

Οδηγίες Εγκατάστασης

(Από το README.md). Αντί για το git clone μπορείτε να κατεβάσετε τον φάκελο του πρότζεκτ και να τον αποσυμπιέσετε

1. Κλωνοποίηση Repository

```
git clone https://github.com/kvelee/mellonbank.git  
cd mellonbank
```

2. (Προαιρετικό, μόνο εάν δεν το έχετε) Pull SQL Server Image

```
docker pull mcr.microsoft.com/mssql/server:2022-latest
```

3. Εκτέλεση Εφαρμογής

1.

```
docker compose up --build db -d
```

2.

```
docker compose up --build web
```

4. Σύνδεση

Συνδέεστε στην εφαρμογή από το

```
localhost:8080
```

Σημαντικό: Ο πρώτος χρήστης που δημιουργείται είναι ο

Username: admin

Password: password